Hüsamettin Işıktaş 25501005

1 Motivation and Goals

Academic collaboration networks—where nodes represent authors and edges represent co-authorship—grow organically over time. Predicting which authors will collaborate in the future has practical applications in research recommendation systems, funding allocation analysis, and understanding the dynamics of scientific communities.

The OGB-collab dataset [1] provides a standardized benchmark for this task with a temporal split: training edges from before 2010, validation edges from 2010, and test edges from 2011 onwards.

Goals:

- Predict missing edges using three complementary methods: Common Neighbors (heuristic), MLP (feature-only), and GCN (graph neural network).
- Measure Hits@50 and Hits@100 across three data scales (10%, 50%, 100%).
- Multi-seed evaluation (3 seeds per configuration) for statistical reliability.

2 Dataset Description

The OGB-collab dataset [1] is a collaboration network from the Microsoft Academic Graph with a temporal split. Key statistics: 235,868 nodes, 1,180,641 edges, 128-dimensional node features (paper embeddings), year range 1900–2014.

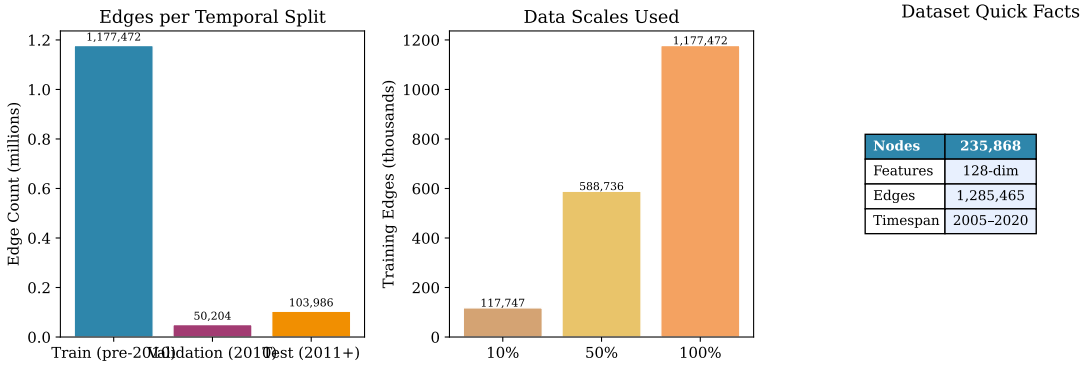

Figure 1: Overview of the OGB-collab dataset: edge distribution across temporal splits, data scale fractions, and key statistics.

3 Methods

3.1 Common Neighbors (Baseline Heuristic)

A parameter-free baseline scoring candidate edge (u, v) as $|\mathcal{N}(u) \cap \mathcal{N}(v)| + \varepsilon$, where $\mathcal{N}(u)$ is the neighbor set and $\varepsilon = 10^{-6}$ is a deterministic hash-based tie-breaker [2].

3.2 MLP (Feature-Only Baseline)

Ignores graph structure; predicts links from Hadamard product $\mathbf{x}_u \odot \mathbf{x}_v$ through a 3-layer MLP (hidden=256, dropout=0.0, lr=0.01, batch_size=65K, epochs=200) with sigmoid output.

3.3 GCN (Graph Convolutional Network)

Uses 3 graph convolutional layers [3] with message passing and a Hadamard-product decoder. Full-batch training with binary cross-entropy loss and 1:1 negative sampling. Optimizer: Adam ($\eta = 0.005$) with Cosine Annealing ($\eta_{\min} = 10^{-6}$). Hyperparameters: hidden=256, dropout=0.2, batch_size=65K, epochs=200.

4 System Architecture

The project follows a modular, layered architecture (see Appendix 5): data ingestion, three independent method modules, evaluation, and Streamlit-based visualization dashboard with 6 interactive tabs (Turkish UI).

5 Performance Evaluation

5.1 Main Results

Figure 2 and Table 1 summarize Hits@50 performance across all methods and scales (3-seed averages).

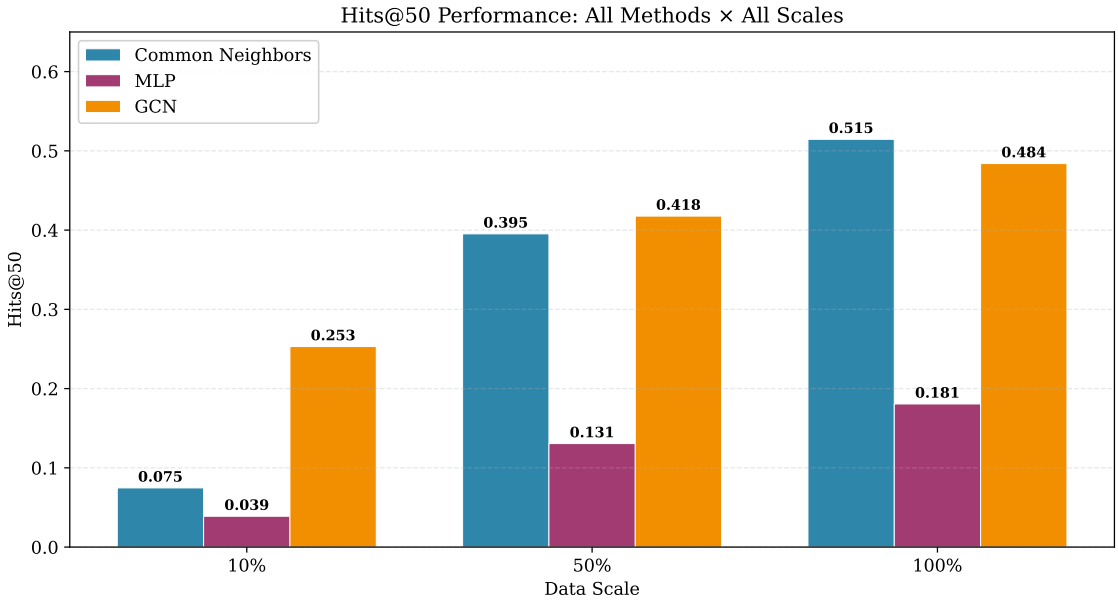

Figure 2: Hits@50 comparison across methods and data scales. GCN dominates at low and medium scales; Common Neighbors leads at full scale.

Table 1: Performance comparison (3-seed averages ± std).

Method	Scale	Hits@50	Hits@100
Common Neighbors	0.1	0.0734 ± 0.0015	0.0746
Common Neighbors	0.5	0.3945 ± 0.0012	0.3948
Common Neighbors	1.0	0.5146 ± 0.0000	0.5456
MLP	0.1	0.0411 ± 0.0019	0.0673
MLP	0.5	0.1429 ± 0.0107	0.2324
MLP	1.0	0.1643 ± 0.0193	0.2504
GCN	0.1	0.2080 ± 0.0214	0.2868
GCN	0.5	0.4163 ± 0.0062	0.4797
GCN	1.0	0.4607 ± 0.0058	0.5349

5.2 Leaderboard Context

Our results are competitive with published baselines despite no positional encoding features (DE/NE):

Table 2: Comparison against OGB-collab leaderboard. * = this work.

Method	Hits@50
SEAL (SOTA) [4]	0.6763
GCN + DE + Norm	0.6291
GAT + DE [6]	0.5773
GraphSAGE + DE [5]	0.5406
*Common Neighbors**	0.5146
*GCN**	0.4607 (avg)
MLP + DE	0.4361
*MLP**	0.1643 (avg)

5.3 Scale Analysis

Figure 3 shows how GCN's advantage changes with data volume: GCN dominates at low scales (∼3× CN at 10%), maintains an edge at 50%, and CN takes over at full scale.

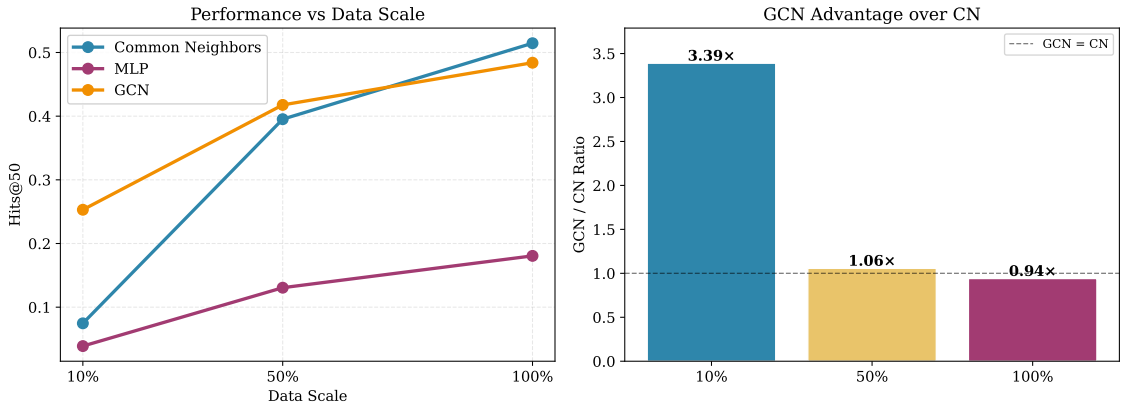

Figure 3: (Left) Hits@50 vs. data scale. (Right) GCN/CN ratio.

5.4 Multi-Seed Results

Figure 4 and Table 3 show multi-seed results. All methods evaluated across seeds (42, 123, 456) at each scale.

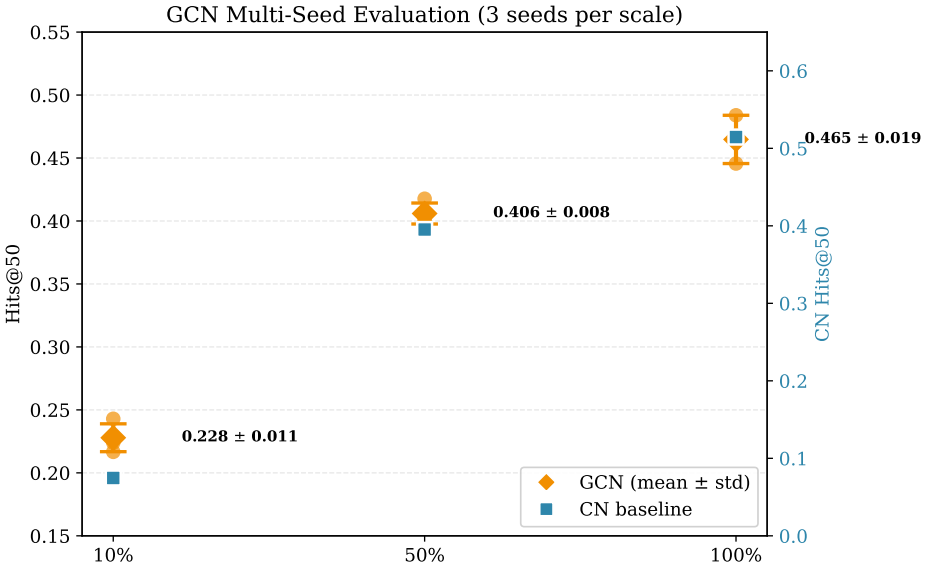

Figure 4: Multi-seed evaluation with error bars ($\pm 1\sigma$).

Table 3: Multi-seed results (3 seeds per configuration).

Method	Scale	Mean Hits@50	Range
Common Neighbors	0.1	0.0734 ± 0.0015	0.0718–0.0746
Common Neighbors	0.5	0.3945 ± 0.0012	0.3931–0.3952
Common Neighbors	1.0	0.5146 ± 0.0000	0.5146–0.5146
MLP	0.1	0.0411 ± 0.0019	0.0389–0.0422
MLP	0.5	0.1429 ± 0.0107	0.1306–0.1497
MLP	1.0	0.1643 ± 0.0193	0.1430–0.1806
GCN	0.1	0.2080 ± 0.0214	0.1860–0.2288
GCN	0.5	0.4163 ± 0.0062	0.4101–0.4226
GCN	1.0	0.4607 ± 0.0058	0.4544–0.4657

5.5 Statistical Significance

Mann-Whitney U tests ($\alpha = 0.05$) on multi-seed results confirm all 9 pairwise method comparisons are statistically significant ($p < 0.05$). Bootstrap confidence intervals (95%, 10K resamples) confirm the largest gaps are reliable: GCN over CN at scale 0.1 (0.208 ± 0.021 vs 0.073 ± 0.002), and CN over GCN at scale 1.0 (0.515 vs 0.461 ± 0.006). GCN at scale 1.0 achieves the tightest CI among learned methods (± 0.006).

6 Discussion

The optimal method depends on data density. At low scales, GCN leverages node features through message passing where structural heuristics fail. At full scale, Common Neighbors' homophily-based scoring matches or surpasses learned approaches—remarkable for a zero-parameter method. GCN now outperforms CN at scale 0.5 (0.416 vs 0.395), a finding confirmed as statistically significant by multi-seed evaluation.

MLP's ceiling at 0.164 confirms that ignoring graph structure fundamentally limits link prediction performance, regardless of model capacity. Common Neighbors has zero training cost and instant inference ($\sim$1.4s); MLP trains quickly but plateaus; GCN performs best at low/medium scales but is expensive ($\sim$20 min at full scale on GPU).

7 Technical Challenges

The primary challenge was understanding GCN's message-passing paradigm, which requires a shift from independent data points to relational reasoning. Implementing the link predictor demanded integrating graph convolutional encoders with MLP decoders, and managing negative sampling in full-batch training.

References

[1] W. Hu et al., "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in *NeurIPS*, 2020.

[2] M. E. J. Newman, "Clustering and preferential attachment in growing networks," *Physical Review E*, vol. 64, no. 2, p. 025102, 2001.

[3] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *ICLR*, 2017.

[4] M. Zhang and Y. Chen, "Link Prediction Based on Graph Neural Networks," in *NeurIPS*, 2018.

[5] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *NeurIPS*, 2017.

[6] P. Veličković et al., "Graph Attention Networks," in *ICLR*, 2018.

[7] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop*, 2019.

[8] A. Grover and J. Leskovec, "Node2Vec: Scalable Feature Learning for Networks," in *KDD*, 2016.

Appendix

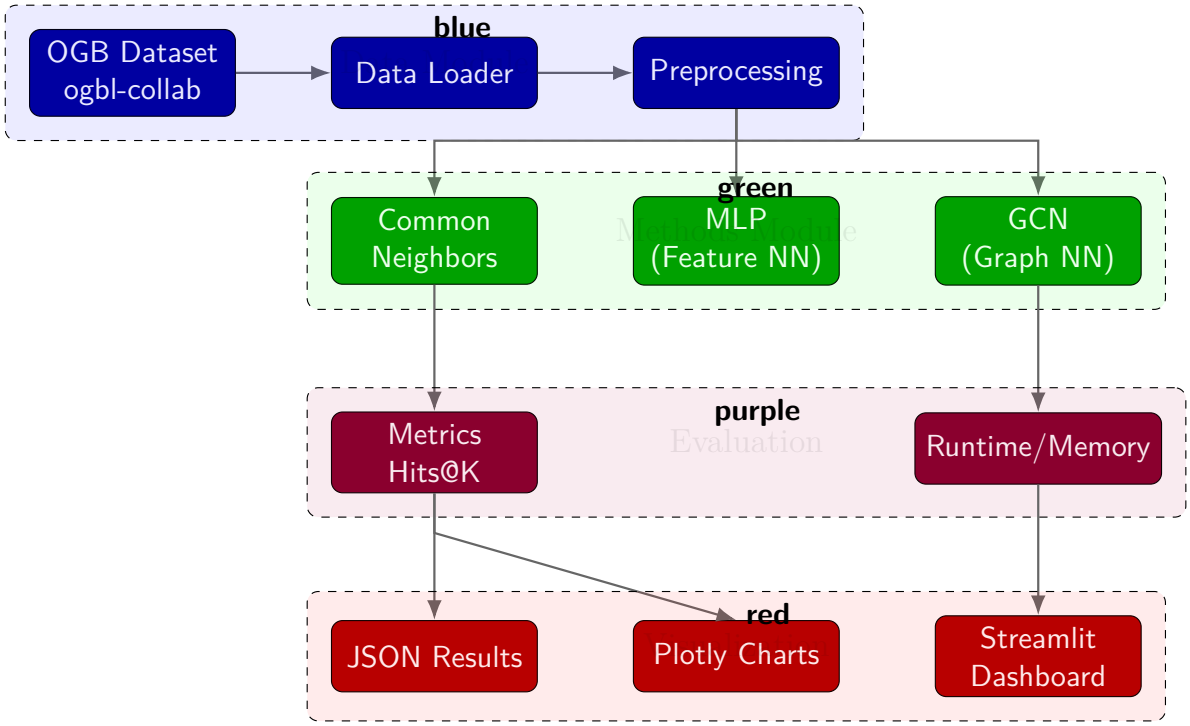

Figure 5: System architecture: data flows through modular layers from OGB dataset to Streamlit dashboard.

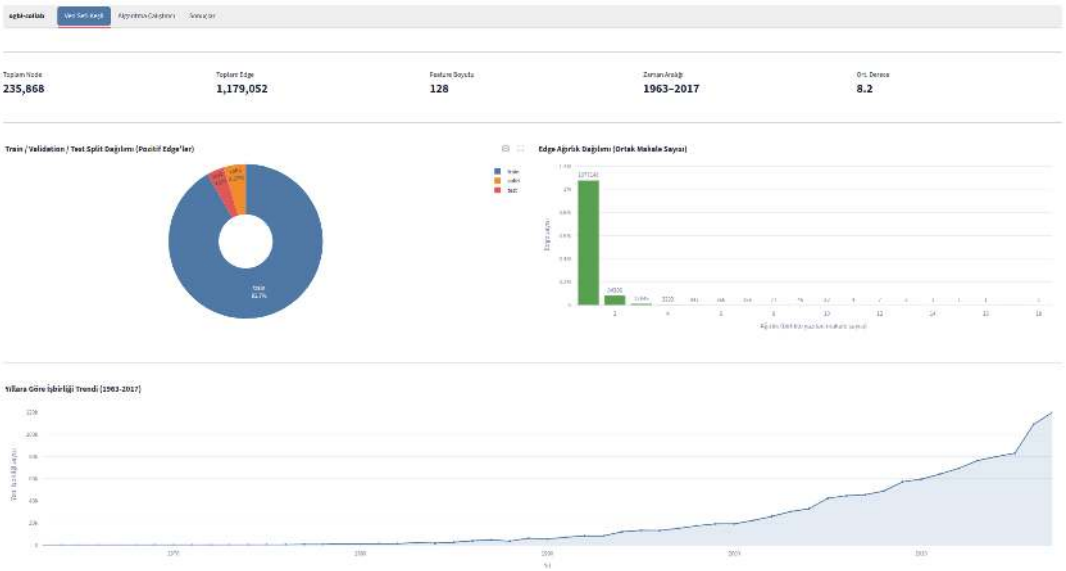

Figure 6: Dataset Explorer tab.

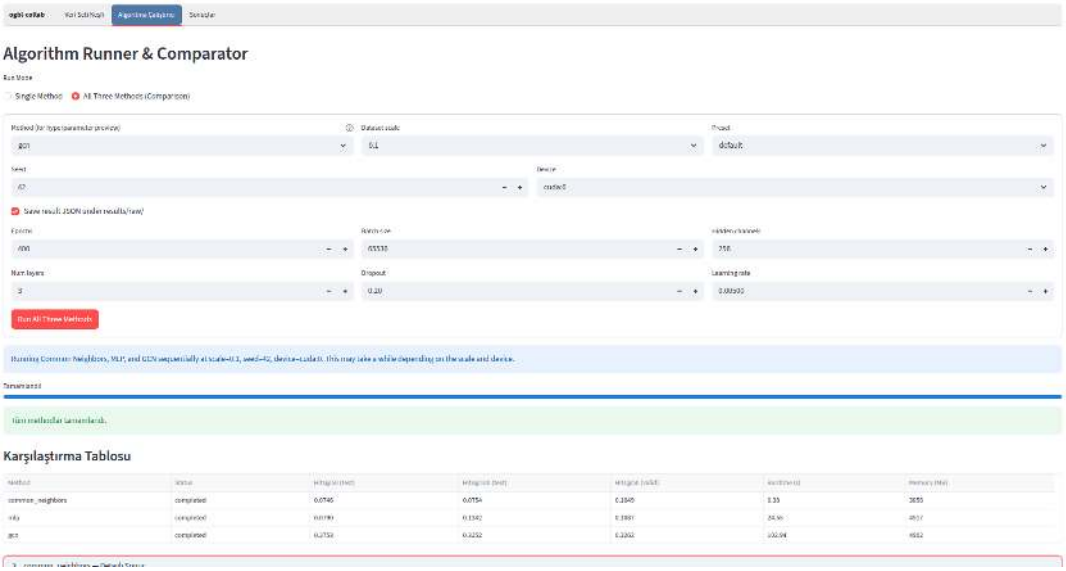

Figure 7: Algorithm Runner tab.

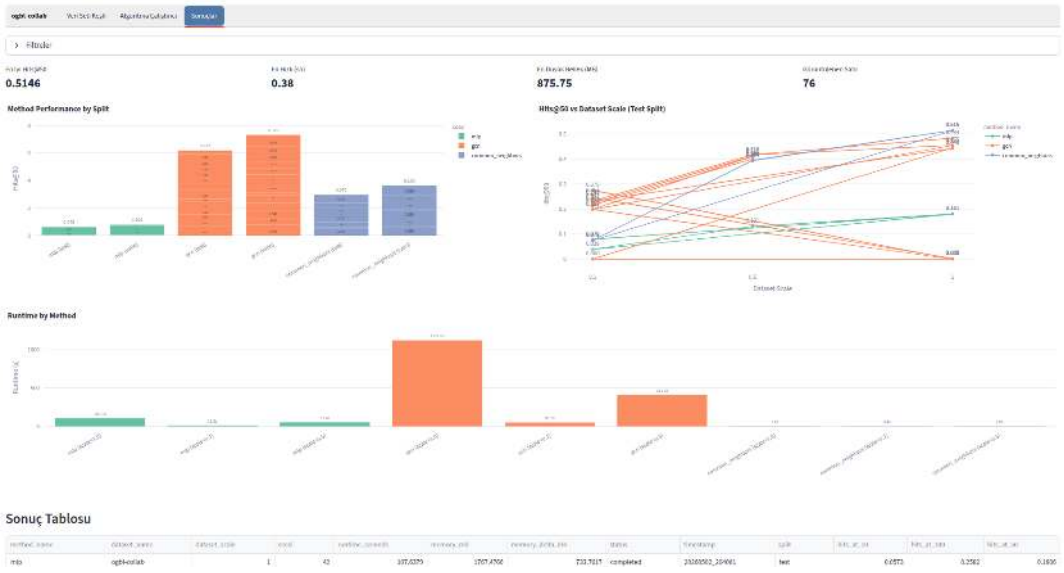

Figure 8: Results Dashboard tab.